



Migrating from Playwright to Cypress

This guide helps you translate Playwright tests to Cypress. It covers the core execution model differences, syntax mappings, and patterns for auth, networking, and CI.

Most migrations are incremental. You do not need to migrate all tests at once. Cypress and Playwright can coexist in the same repository during a transition.

Jump to the [cheat sheet](#) for a quick reference.

? What you'll learn

- How Cypress executes tests: command chaining, retry-ability, and what replaces `async/await`
- Configuration, CLI flags, and browser installation
- Locators, interactions, assertions, and network stubbing mapped side by side
- Authentication, cross-origin testing, and multi-tab patterns
- Clock control, dialogs, iframes, and file handling
- Page objects, parameterized tests, and custom commands
- Parallelization, Smart Orchestration, Test Replay, and accessibility testing in Cypress Cloud

Quick conversion example

Here's a side-by-side comparison showing how Playwright tests translate to Cypress:

Before: Playwright



test.ts

```
import { test, expect } from '@playwright/test'

test.describe('Authorization', () => {
  test('signs up', async ({ page }) => {
    // Navigate
    await page.goto('/signup')

    // Fill form
    await page.getByLabel('Email').fill('user@email.com')
    await page.getByLabel('Confirm
Email').fill('user@email.com')
    await page.getByLabel('Password').fill('testPassword1234')

    // Submit
    await page.getByRole('button', { name: 'Create new account'
}).click()

    // Assert navigation
    await expect(page).toHaveURL(/\/signup\/success$/)
  })
})
```

After: Cypress**test.cy.ts**

```
describe('Authorization', () => {
  it('signs up', () => {
    // Navigate
    cy.visit('/signup')

    // Fill form
    cy.get('[data-testid="email"]').type('user@email.com')
    cy.get('[data-testid="confirm-
email"]').type('user@email.com')
    cy.get('[data-testid="password"]').type('testPassword1234')
```



```
// Submit
cy.contains('button', 'Create new account').click()

// Assert navigation
cy.url().should('include', 'signup/success')
})
})
```

Installing Cypress

Before migrating your tests, add Cypress to your project as a dev dependency.

npm yarn pnpm

```
npm install cypress --save-dev
```

Unlike Playwright, Cypress does not download its own browser binaries. It uses browsers already installed on your machine. Electron is bundled with Cypress and available by default, so you can run tests immediately after installation without any additional setup. Once installed, open the Cypress app to complete initial configuration:

npm yarn pnpm

```
npx cypress open
```

The Cypress App will walk you through choosing a testing type (end-to-end or component) and generating a starter configuration file. After setup, Cypress creates a `cypress.config.js` (or `.ts`) file and a `cypress/` directory in your project root.

Configuration migration

Both Playwright and Cypress use a `defineConfig`-based configuration file at the root of your project. The key structural difference is that Cypress scopes most E2E-specific options under an `e2e` object, rather than a flat `use` block.



Config file structure

The configuration file location is configurable using the `--config-file` command line flag or the `configFile` Module API option.

	Playwright	Cypress
Default config file	<code>playwright.config.ts</code>	<code>cypress.config.ts</code>
Test files	<code>testDir: './tests'</code>	<code>e2e.specPattern</code> (glob)
E2E-specific options	<code>use: { ... }</code>	<code>e2e: { ... }</code>
Node event hooks	<code>globalSetup</code> / <code>globalTeardown</code>	<code>e2e.setupNodeEvents</code>

Before: Playwright

playwright.config.ts

```
import { defineConfig } from '@playwright/test'

export default defineConfig({
  testDir: './tests',
  testExclude: '**/*.cy.ts',
  outputDir: './test-results',
  retries: process.env.CI ? 2 : 0,
  timeout: 30000,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:3000',
    viewport: { width: 1280, height: 720 },
    video: 'off',
    screenshot: 'only-on-failure',
  },
})
```

After: Cypress**cypress.config.ts**

```
import { defineConfig } from 'cypress'

export default defineConfig({
  viewportWidth: 1280,
  viewportHeight: 720,
  video: false,
  videosFolder: './test-results/videos',
  reporter: 'html',
  screenshotsFolder: './test-results/screenshots',
  e2e: {
    baseUrl: 'http://localhost:3000',
    specPattern: 'tests/**/*.cy.ts',
    excludeSpecPattern: '**/*.ts',
    retries: {
      runMode: 2,
      openMode: 0,
    },
    defaultCommandTimeout: 30000,
    screenshotOnRunFailure: true,
  },
})
```

See [Cypress Configuration](#) for the full option reference.

Common translations

Playwright	Cypress	Notes
<code>forbidOnly</code>	use <code>mocha/no-exclusive-tests</code> lint rule	eslint-plugin-mocha
<code>headless</code>	<code>--headless</code> or <code>--headed</code> CLI flag	Default headless in <code>cypress run</code>



Playwright	Cypress	Notes
<code>outputDir</code>	<code>screenshotsFolder</code> / <code>videosFolder</code>	Separate options per asset type
<code>reporter</code>	<code>reporter</code>	See Reporters .
<code>retries</code>	<code>retries.runMode</code> / <code>retries.openMode</code>	Cypress separates CI and interactive mode retries
<code>testDir</code>	<code>specPattern</code>	Cypress uses a glob, not a path.
<code>testIgnore</code>	<code>excludeSpecPattern</code>	Glob string or array of globs.
<code>timeout</code>	<code>defaultCommandTimeout</code>	<code>defaultCommandTimeout</code> is per command, not test.
<code>use.baseUrl</code>	<code>e2e.baseUrl</code>	Must be under <code>e2e</code> , not top-level
<code>use.channel</code>	<code>--browser <name>: <channel></code> on the CLI	See Launching Browsers .
<code>use.proxy</code>	OS environment variables	See Proxy Configuration
<code>use.screenshot</code>	<code>screenshotOnRunFailure</code>	Default is <code>true</code>
<code>use.video</code>	<code>video</code>	Default is <code>false</code>
<code>use.viewport</code>	<code>viewportWidth</code> / <code>viewportHeight</code>	Set at top level or scoped under testing type

CLI/Command line migration



Playwright and Cypress use different commands and flag structures. This section covers the most common translations.

Base commands

Task	Playwright	Cypress
Run all tests (headless)	<code>npx playwright test</code>	<code>cypress run</code>
Open interactive mode	<code>npx playwright test --ui</code>	<code>cypress open</code>
Run in headed mode	<code>npx playwright test --headed</code>	<code>cypress run --headed</code>

Common flag translations

Playwright	Cypress
<code>--config=playwright.config.ts</code>	<code>--config-file cypress.config.ts</code>
<code>npx playwright test todo.spec.ts</code>	<code>cypress run --spec "todo.cy.ts"</code>
<code>--fail-on-flaky-tests</code>	<code>detect-flake-but-always-fail</code> in config with Experimental Test Retries
<code>--forbid-only</code>	<code>mocha/no-exclusive-tests</code> eslint-plugin-mocha rule
<code>--grep "pattern"</code>	<code>--expose grep="pattern"</code> from @cypress/grep



Playwright	Cypress
<code>--grep-invert "pattern"</code>	<code>--expose grep="-pattern"</code> from @cypress/grep
<code>--last-failed</code>	Spec Prioritization
<code>--max-failures=5</code>	<code>--auto-cancel-after-failures=5</code>
<code>--project=chromium</code>	<code>--project='./front-end'</code>
<code>--quiet</code>	<code>--quiet</code>
<code>--retries=2</code>	<code>--config retries=2</code>
<code>--reporter=html</code>	<code>--reporter html</code>
<code>--shard=1/4</code>	<code>--parallel</code>
<code>--fully-parallel</code>	<code>--parallel</code>
<code>--workers=4</code>	<code>--parallel</code>
<code>--trace on</code>	<code>--record</code> automatically records Test Replay

Passing environment variables

In Cypress, use `cy.env()` for sensitive values (credentials, tokens, secrets) or `Cypress.expose()` for non-sensitive configuration (feature flags, public URLs, API versions). These must be set in the Node.js environment within `setupNodeEvents` in your Cypress configuration file.

Before: Playwright

```
test.ts
```



```
test('logs in', async ({ page }) => {  
  await page.getByLabel('User  
Name').fill(process.env.USER_NAME)  
  await page.getByLabel('Password').fill(process.env.PASSWORD)  
})
```

After: Cypress

cypress.config.ts

```
export default defineConfig({  
  expose: {  
    API_VERSION: 'v2',  
  },  
  env: {  
    USER_NAME: process.env.USER_NAME,  
    PASSWORD: process.env.PASSWORD,  
  },  
})
```

test.cy.ts

```
it('logs in', () => {  
  cy.env(['USER_NAME', 'PASSWORD']).then(({ USER_NAME, PASSWORD  
})) => {  
    const apiVersion = Cypress.expose('API_VERSION')  
    cy.visit(`/login/${apiVersion}`)  
    cy.get('[data-testid="username"]').type(USER_NAME)  
    cy.get('[data-testid="password"]').type(PASSWORD)  
  })  
})
```

See [cy.env\(\)](#) and [Cypress.expose\(\)](#) for the full reference.

Browser installation



Playwright and Cypress take different approaches to browser management. Understanding the difference will help you set up CI correctly.

How each framework handles browsers

Playwright downloads and manages its own isolated browser binaries. Running `npx playwright install` fetches Playwright-specific builds of Chromium, Firefox, and WebKit that are separate from any browsers installed on your system.

Cypress uses browsers already installed on the machine. There are no browser binaries to download. If Chrome is installed locally, Cypress will find and use it. This means browser setup is a CI concern rather than a per-project dependency.

Before: Playwright

```
npx playwright install chromium webkit
```

After: Cypress

To verify which browsers Cypress can find:

```
cypress info
```

Browser versions and CI

Because Cypress uses system browsers, you have explicit control over which browser version your tests run against.

Docker images

Cypress publishes official Docker images with specific browser versions pre-installed. Image tags encode the exact versions of Node, Chrome, Firefox, and Edge, so your CI environment is reproducible and version changes are explicit.

Node + browsers, without Cypress

```
cypress/browsers:node-22.19.0-chrome-139.0.7258.154-1-ff-142.0.1-edge-139.0.3405.125-1
```

Node + browsers + Cypress pre-installed



```
cypress/included:cypress-15.1.0-node-22.19.0-chrome-139.0.7258.154-1-ff-142.0.1-edge-139.0.3405.125-1
```

Use `cypress/browsers` when you want to control the Cypress version through your `package.json`. Use `cypress/included` when you want a fully self-contained image. If you need a specific combination of versions not covered by the published images, use `cypress/factory` to build a custom image.

See [cypress-docker-images](#) for all available tags.

CircleCI orb

If you use CircleCI, the [Cypress CircleCI orb](#) handles installation, caching, and test execution. Pass `install-browsers: true` to install Chrome, Firefox, and Edge into the CI environment.

`.circleci/config.yml`

```
version: 2.1
orbs:
  cypress: cypress-io/cypress@6
workflows:
  build:
    jobs:
      - cypress/run:
          install-browsers: true
          start-command: 'npm start'
          cypress-command: 'npx cypress run --browser chrome'
```

See [CircleCI](#) for full setup documentation.

Browser launch options

Playwright's `launchOptions` in `playwright.config.ts` maps to the `before:browser:launch` event in Cypress, which you configure inside `setupNodeEvents` of your Cypress configuration file. Both let you modify how the browser starts. See the [Browser Launch Event](#) for the full reference.



Launch arguments, extensions, and preferences

Cypress exposes the `before:browser:launch` event.

Before: Playwright

playwright.config.ts

```
export default defineConfig({
  use: {
    launchOptions: {
      args: ['--disable-gpu'],
    },
  },
})
```

After: Cypress

cypress.config.ts

```
export default defineConfig({
  e2e: {
    setupNodeEvents(on) {
      on('before:browser:launch', (browser, launchOptions) => {
        if (browser.family === 'chromium' && browser.name !==
        'electron') {
          launchOptions.args.push('--disable-gpu')
        }
        return launchOptions
      })
    },
  },
})
```

The `launchOptions` object exposes three properties depending on the browser:



Capability	Playwright	Cypress
Browser launch arguments	<code>launchOptions.args</code>	<code>launchOptions.args</code> via <code>before:browser:launch</code>
Firefox preferences	<code>launchOptions.firefoxUserPrefs</code>	<code>launchOptions.preferences</code> via <code>before:browser:launch</code>
Chrome/Edge extensions	<code>args</code> flags in <code>launchPersistentContext()</code>	<code>launchOptions.extensions</code> via <code>before:browser:launch</code>

See the [Browser Launch Event](#) for the full reference and per-browser examples.

Test structure and syntax migration

Playwright imports `test` from `@playwright/test`. Cypress uses Mocha's BDD interface, which provides `it()` and `describe()` globally. `test` is not available in Cypress and must be replaced.

Playwright	Cypress
<code>test('name', () => {})</code>	<code>it('name', () => {})</code>
<code>test.only('name', () => {})</code>	<code>it.only('name', () => {})</code>
<code>test.skip('name', () => {})</code>	<code>it.skip('name', () => {})</code>
<code>test.describe('group', () => {})</code>	<code>describe('group', () => {})</code>
<code>test.describe.only('group', () => {})</code>	<code>describe.only('group', () => {})</code>

Before: Playwright



test.ts

```
import { test, expect } from '@playwright/test'

test.describe('checkout', () => {
  test('completes purchase', async ({ page }) => { ... })
  test.only('applies discount', async ({ page }) => { ... })
  test.skip('handles expired card', async ({ page }) => { ...
})
})
```

After: Cypress

test.cy.ts

```
describe('checkout', () => {
  it('completes purchase', () => { ... })
  it.only('applies discount', () => { ... })
  it.skip('handles expired card', () => { ... })
})
```

Execution model and retry-ability

Playwright code typically awaits each action and may use explicit waits for specific conditions. Cypress commands are enqueued and automatically retry assertions until they pass or timeout.

What this means for your tests:

Playwright	Cypress
<code>await</code> <code>page.waitForSelector(...)</code>	<code>cy.get(...)</code> retries until it finds the element
<code>await expect(locator)...</code>	<code>cy.get(...).should(...)</code> retries automatically



Before: Playwright**test.ts**

```
await page.getByTestId('submit').click()
await page.waitForSelector('[data-testid="toast"]')
await expect(page.getByTestId('toast')).toHaveText('Saved!')
```

After: Cypress**test.cy.ts**

```
cy.get('[data-testid="submit"]').click()
cy.get('[data-testid="toast"]').should('have.text', 'Saved!')
```

Learn more about Cypress [retry-ability](#).

Async/await vs command chaining

Playwright uses async/await patterns.

Before: Playwright**test.ts**

```
const button = await page.getByRole('button', { name: 'Submit' })
await button.click()
const toast = await page.getByTestId('toast')
await expect(toast).toHaveText('Success')
```

Cypress uses command chaining without async/await.

After: Cypress**test.cy.ts**

```
cy.get('button').contains('Submit').click()  
cy.get('[data-testid="toast"]').should('have.text', 'Success')
```

See Cypress's command model in the [Introduction to Cypress](#).

Locators and selectors

Prefer stable selectors

Use a stable selector strategy (often `data-*`) as described in Cypress [Best Practices](#).

Before: Playwright

test.ts

```
await page.getByTestId('email').fill('user@email.com')  
await page.getByRole('button', { name: 'Sign in' }).click()
```

After: Cypress

test.cy.ts

```
cy.get('[data-testid="email"]').type('user@email.com')  
cy.contains('button', 'Sign in').click()
```

getByRole() and getLabel()

If you want `getByRole()`/`getLabel()` ergonomics in Cypress, install [Cypress Testing Library](#).

Before: Playwright

test.ts

```
await page.getByRole('button', { name: 'Save' }).click()
```




```
await page.getByLabel('First name').fill('Jane')
```

After: Cypress

test.cy.ts

```
cy.findByRole('button', { name: 'Save' }).click()  
cy.findByLabelText('First name').type('Jane')
```

Interactions

Click, type, and clear

Before: Playwright

test.ts

```
await page.locator('#username').fill('jane')  
await page.locator('#password').fill('secret')  
await page.getByRole('button', { name: 'Log in' }).click()
```

After: Cypress

test.cy.ts

```
cy.get('#username').clear()  
cy.get('#username').type('jane')  
cy.get('#password').clear()  
cy.get('#password').type('secret')  
cy.contains('button', 'Log in').click()
```

Checkboxes, radios, and selects

Before: Playwright

test.ts



```
await page.getByLabel('Subscribe').check()  
await page.getByLabel('Daily').check()  
await page.getByLabel('Country').selectOption('US')
```

After: Cypress

test.cy.ts

```
cy.get('[data-testid="subscribe"]').check()  
cy.get('[data-testid="daily"]').check()  
cy.get('[data-testid="country"]').select('US')
```

Hover

Before: Playwright

test.ts

```
await page.getByTestId('menu').hover()  
await page.getByRole('menuitem', { name: 'Settings' }).click()
```

After: Cypress

test.cy.ts

```
cy.get('[data-testid="menu"]').trigger('mouseover')  
cy.contains('[role="menuitem"]', 'Settings').click()
```

Mobile viewport testing

Before: Playwright

test.ts

```
await page.setViewportSize({ width: 375, height: 812 })  
await page.goto('/')
```



After: Cypress**test.cy.ts**

```
cy.viewport(375, 812)
cy.visit('/')
```

See `cy.viewport()`.

Assertions

Visibility and existence

Before: Playwright**test.ts**

```
await expect(page.getByText('Welcome')).toBeVisible()
await expect(page.getByTestId('spinner')).toBeHidden()
```

After: Cypress**test.cy.ts**

```
cy.contains('Welcome').should('be.visible')
cy.get('[data-testid="spinner"]').should('not.be.visible')
```

Text, attributes, and URL

Before: Playwright**test.ts**

```
await expect(page.getByTestId('toast')).toHaveText('Saved!')
await expect(page.getByRole('button', { name: 'Save'
```



```
))).toBeDisabled()  
await expect(page).toHaveURL(/\\/dashboard$/)
```

After: Cypress**test.cy.ts**

```
cy.get('[data-testid="toast"]').should('have.text', 'Saved!')  
cy.contains('button', 'Save').should('be.disabled')  
cy.url().should('match', /\\/dashboard$/)
```

See Cypress assertions at [Assertions](#).

Waiting, retries, and timeouts

Playwright often uses explicit waits. Cypress will retry most DOM queries and assertions until they pass or time out.

Before: Playwright**test.ts**

```
await page.getByRole('button', { name: 'Save' }).click()  
await page.waitForResponse('**/api/profile')  
await expect(page.getByTestId('toast')).toHaveText('Saved!')
```

After: Cypress**test.cy.ts**

```
cy.intercept('POST', '/api/profile').as('saveProfile')  
cy.contains('button', 'Save').click()  
cy.wait('@saveProfile')  
cy.get('[data-testid="toast"]').should('have.text', 'Saved!')
```

Learn more about Cypress retry-ability at [Retry-ability](#).



Network spying and stubbing

Spy

Before: Playwright

test.ts

```
const [response] = await Promise.all([
  page.waitForResponse('**/api/users'),
  page.getByRole('button', { name: 'Load users' }).click(),
])
expect(response.ok()).toBeTruthy()
```

After: Cypress

test.cy.ts

```
cy.intercept('GET', '/api/users').as('users')
cy.contains('button', 'Load users').click()
cy.wait('@users').its('response.statusCode').should('eq', 200)
```

Stub a response

Before: Playwright

test.ts

```
await page.route('**/api/projects', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([
      { id: '1' },
      { id: '2' }
    ]),
  })
})
```

After: Cypress



test.cy.ts

```
cy.intercept('GET', '/api/projects', {
  statusCode: 200,
  body: [{ id: '1' }, { id: '2' }],
}).as('projects')
```

Modify a response

Before: Playwright**test.ts**

```
await page.route('**/api/profile', async (route) => {
  const response = await route.fetch()
  const json = await response.json()
  json.plan = 'enterprise'
  await route.fulfill({ response, json })
})
```

After: Cypress**test.cy.ts**

```
cy.intercept('GET', '/api/profile', (req) => {
  req.continue((res) => {
    res.body.plan = 'enterprise'
  })
}).as('profile')
```

See [Network Requests](#) and `cy.intercept()`.

Authentication patterns

Playwright commonly reuses auth via `storageState`. In Cypress, use `cy.session()` to cache cookies and storage between tests.



Before: Playwright**playwright.config.ts**

```
import { defineConfig } from '@playwright/test'

export default defineConfig({
  use: {
    storageState: 'storageState.json',
  },
})
```

auth.setup.ts

```
import { test as setup } from '@playwright/test'

setup('authenticate', async ({ page }) => {
  await page.goto('/login')
  await page.getByLabel('Username').fill('jane')
  await page.getByLabel('Password').fill('secret')
  await page.getByRole('button', { name: 'Log in' }).click()

  await page.context().storageState({ path: 'storageState.json' })
})
```

After: Cypress**cypress.config.ts**

```
const login = () => {
  cy.visit('/login')
  cy.get('#username').type('jane')
  cy.get('#password').type('secret')
  cy.contains('button', 'Log in').click()
}

beforeEach(() => {
```



```
cy.session('jane', login)
})
```

Cross-origin

Cypress enforces a same-origin policy by default. Use `cy.origin()` to run commands against a different origin within the same test. Any commands targeting a secondary origin must be scoped inside the `cy.origin()` callback.

Before: Playwright

test.ts

```
await page.goto('https://site-a.example')
await page.goto('https://site-b.example')
await expect(page.getByRole('banner')).toBeVisible()
```

After: Cypress

test.cy.ts

```
cy.visit('https://site-a.example')
cy.origin('https://site-b.example', () => {
  cy.visit('https://site-b.example')
  cy.get('[role="banner"]').should('be.visible')
})
```

See `cy.origin()` for complete documentation.

Screenshots

Before: Playwright

```
await page.screenshot({ path: 'after-signup.png' })
await page.locator('[data-testid="toast"]').screenshot({ path:
'toast.png' })
```

After: Cypress




```
cy.screenshot('after-signup')  
cy.get('[data-testid="toast"]').screenshot('toast')
```

See [Screenshots and Videos](#).

File uploads and downloads

Upload

Before: Playwright

```
await page.setInputFiles('input[type="file"]',  
  'fixtures/avatar.png')
```

After: Cypress

```
cy.get('input[type="file"]').selectFile('cypress/fixtures/avatar.png')
```

See [cy.selectFile\(\)](#).

Downloads

In Cypress, a common approach is to trigger the download, then assert the file exists in the configured downloads folder.

Before: Playwright

```
const download = await page.waitForEvent('download')  
await page.getByRole('button', { name: 'Export CSV' }).click()  
const path = await download.path()
```

After: Cypress

```
cy.contains('button', 'Export CSV').click()  
cy.readFile('cypress/downloads/export.csv')
```



See `cy.readFile()`.

Clock and time control

Both Playwright and Cypress let you replace native browser time functions with a fake clock that you control. This allows tests to set a specific date, skip waiting for timers, and test time-sensitive UI behavior without real delays. The two tools take different approaches to time control.

Before: Playwright

test.ts

```
await page.clock.setFixedTime(new Date('2024-02-02T10:00:00'))
await
page.goto('/')
await
expect(page.getByTestId('current-time')).toHaveText('2/2/2024,
10:00:00 AM')
```

After: Cypress

test.cy.ts

```
cy.clock(new Date('2024-02-02T10:00:00')) cy.visit('/')
cy.get('[data-testid="current-time"]').should('have.text',
'2/2/2024, 10:00:00
AM')
```

Advancing time

To simulate the passage of time and trigger scheduled timers, Playwright uses `fastForward` or `runFor`. Cypress uses `cy.tick()`.

Before: Playwright

test.ts

```
await page.clock.install()
await page.goto('/')
await
page.getByRole('button').click()
```



```
// Skip forward 5 minutes, firing all timers that fall within
the window
await page.clock.fastForward('05:00')

await expect(page.getByText('Session expired')).toBeVisible()
```

After: Cypress

test.cy.ts

```
cy.clock()
cy.visit('/')
cy.getByRole('button').click()

// cy.tick() takes milliseconds; 5 minutes = 300,000 ms
cy.tick(5 * 60 * 1000)

cy.contains('Session expired').should('be.visible')
```

Pausing at a specific time

Playwright supports `pauseAt`, which lets the clock run naturally from its installed time and then pause automatically when it reaches a target timestamp. For Cypress to reach a specific point in time, calculate the delta between the installed time and the target time and pass that to `cy.tick()`.

Before: Playwright

test.ts

```
await page.clock.install({ time: new Date('2024-02-02T08:00:00') })
await page.goto('/')

// Clock runs naturally until it reaches 10:00 AM, then pauses
await page.clock.pauseAt(new Date('2024-02-02T10:00:00'))
await expect(page.getByTestId('current-time')).toHaveText(
  '2/2/2024, 10:00:00 AM'
```



```
)

// Advance another 30 minutes manually
await page.clock.fastForward('30:00')
await expect(page.getByTestId('current-time')).toHaveText(
  '2/2/2024, 10:30:00 AM'
)
```

After: Cypress

test.cy.ts

```
cy.clock(new Date('2024-02-02T08:00:00'))
cy.visit('/')

// Advance 2 hours (from 08:00 to 10:00) manually
cy.tick(2 * 60 * 60 * 1000)
cy.get('[data-testid="current-time"]').should(
  'have.text',
  '2/2/2024, 10:00:00 AM'
)

// Advance another 30 minutes
cy.tick(30 * 60 * 1000)
cy.get('[data-testid="current-time"]').should(
  'have.text',
  '2/2/2024, 10:30:00 AM'
)
```

Changing the system time mid-test

Both tools allow you to change the current time without triggering timers. In Playwright this is `page.clock.setSystemTime()`. In Cypress it is `clock.setSystemTime()` on the yielded clock object.

Before: Playwright

test.ts



```
await page.clock.install({ time: new Date('2024-02-02T10:00:00') })
await page.goto('/')

// Jump to a new time without firing any scheduled timers
await page.clock.setSystemTime(new Date('2024-02-02T10:30:00'))
```

After: Cypress

test.cy.ts

```
cy.clock(new Date('2024-02-02T10:00:00'))
cy.visit('/')

cy.clock().then((clock) => {
  // Jump to a new time without firing any scheduled timers
  clock.setSystemTime(new Date('2024-02-02T10:30:00'))
})
```

Overriding only specific functions

Cypress allows you to limit which native functions the clock replaces. This is useful when you only need to control `Date` without affecting timer behavior, or vice versa.

```
// Override only Date, leave setTimeout and setInterval running normally
cy.clock(Date.UTC(2024, 1, 2), ['Date'])

// Override only timers, leave Date as the system clock
cy.clock(null, ['setTimeout', 'clearTimeout', 'setInterval', 'clearInterval'])
```

Multiple tabs and windows



You can handle multi-tab workflows using the `@cypress/puppeteer` plugin, which gives Cypress access to the underlying browser via Puppeteer. This approach requires Puppeteer as a dependency.

Before: Playwright

test.ts

```
// Start waiting for the new tab before clicking.
const newPagePromise = page.context().waitForEvent('page')
await page.getByText('open new tab').click()
const newPage = await newPagePromise

await newPage.waitForLoadState()
await newPage.getByRole('button', { name: 'Confirm' }).click()
```

After: Cypress

cypress.config.ts

```
import { defineConfig } from 'cypress'
import { setup, retry } from '@cypress/puppeteer'

export default defineConfig({
  e2e: {
    setupNodeEvents(on) {
      setup({
        on,
        onMessage: {
          async switchToTabAndGetContent(browser) {
            const page = await retry(async () => {
              const pages = await browser.pages()
              const page = pages.find((p) =>
                p.url().includes('page-2.html'))
              if (!page) throw new Error('Could not find page')
              return page
            })

            await page.bringToFront()
            const paragraph = await page.waitForSelector('p')
```



```

        return await page.evaluate((el) => el.textContent,
        paragraph)
      },
    },
  })
},
},
})

```

In your support file, import the command once:

```

// cypress/support/e2e.ts
import '@cypress/puppeteer/support'

```

test.cy.ts

```

cy.visit('/')
cy.contains('button', 'Open new tab').click()

cy.puppeteer('switchToTabAndGetContent').should(
  'equal',
  'You said: Hello from Page 1'
)

```

Iframes

Cypress has no equivalent single command. The standard approach is to get the iframe element, access its `contentDocument.body`, and then wrap that body element with `cy.wrap()` so you can use Cypress commands on its contents. The `.should('not.be.empty')` assertion ensures Cypress waits for the frame's document to load before proceeding.

Before: Playwright

test.ts



```
// Locate and fill a field inside an iframe
const frame = page.frameLocator('#payment-frame')
await frame.getByLabel('Card number').fill('4242424242424242')
await frame.getByLabel('Expiry').fill('12/26')
await frame.getByRole('button', { name: 'Pay now' }).click()
```

After: Cypress

test.cy.ts

```
cy.get('#payment-frame')
  .its('0.contentDocument.body')
  .should('not.be.empty')
  .then(cy.wrap)
  .within(() => {
    cy.get('input[aria-label="Card number"]').type('4242424242424242')
    cy.get('input[aria-label="Expiry"]').type('12/26')
    cy.contains('button', 'Pay now').click()
  })
```

If you interact with the same iframe across multiple tests, extract the frame access into a custom command to avoid repeating the boilerplate:

cypress/support/commands.ts

```
Cypress.Commands.add('getIframeBody', (selector: string) => {
  return cy
    .get(selector)
    .its('0.contentDocument.body')
    .should('not.be.empty')
    .then(cy.wrap)
})

// In your test
cy.getIframeBody('#payment-frame').within(() => {
  cy.contains('button', 'Pay now').click()
})
```



Shadow DOM

For Shadow DOM in Cypress, you can traverse explicitly:

Cypress

test.cy.ts

```
cy.get('[data-testid="shadow-host"]').shadow().find('button').click()
```

See `.shadow()`.

Dialogs

alert()

Cypress always accepts `alert()` dialogs.

Before: Playwright

test.ts

```
// Register a handler to inspect the alert text before
accepting
page.on('dialog', async (dialog) => {
  expect(dialog.type()).toBe('alert')
  expect(dialog.message()).toBe('Unsaved changes will be
lost.')
  await dialog.accept()
})

await page.getByRole('button', { name: 'Leave page' }).click()
```

After: Cypress

test.cy.ts



```
// Cypress auto-accepts alerts. Listen to assert on the
message.
cy.on('window:alert', (message) => {
  expect(message).to.eq('Unsaved changes will be lost.')
})

cy.get('button').contains('Leave page').click()
```

confirm()

Before: Playwright

test.ts

```
// Accept the confirmation
page.on('dialog', (dialog) => dialog.accept())
await page.getByRole('button', { name: 'Delete account'
}).click()
await expect(page.getByText('Account deleted')).toBeVisible()

// Dismiss the confirmation
page.on('dialog', (dialog) => dialog.dismiss())
await page.getByRole('button', { name: 'Delete account'
}).click()
await expect(page.getByText('Account
deleted')).not.toBeVisible()
```

After: Cypress

test.cy.ts

```
// Cypress auto-accepts confirm(). No handler needed to test
the accepted path.
cy.get('button').contains('Delete account').click()
cy.contains('Account deleted').should('be.visible')

// Return false to dismiss the confirmation
cy.on('window:confirm', () => false)
```



```
cy.get('button').contains('Delete account').click()
cy.contains('Account deleted').should('not.exist')
```

If you need to assert on the confirmation message text and control acceptance in the same test, combine both:

```
cy.on('window:confirm', (message) => {
  expect(message).to.eq('Are you sure you want to delete your
  account?')
  return false // return true or omit the return to accept
})

cy.get('button').contains('Delete account').click()
```

For tests that need to verify the stub was called, use `cy.stub()` instead of `cy.on()`. This lets you use Sinon assertions to confirm the dialog was triggered the expected number of times.

```
const confirmStub =
cy.stub().as('confirmDialog').returns(false)
cy.on('window:confirm', confirmStub)

cy.get('button').contains('Delete account').click()
cy.get('@confirmDialog').should('have.been.calledOnce')
```

prompt()

In Cypress, you must stub `window.prompt` before the page loads using the `onBeforeLoad` callback in `cy.visit()`. This gives you full control over the return value and lets you assert that the prompt was called.

Before: Playwright

test.ts

```
page.on('dialog', async (dialog) => {
  expect(dialog.type()).toBe('prompt')
  expect(dialog.message()).toBe('Enter your name:')
})
```



```
    await dialog.accept('Ada Lovelace')
  })

  await page.getByRole('button', { name: 'Set name' }).click()
  await expect(page.getByTestId('greeting')).toHaveText('Hello,
  Ada Lovelace!')
```

After: Cypress

test.cy.ts

```
cy.visit('/', {
  onBeforeLoad(win) {
    cy.stub(win, 'prompt').as('promptDialog').returns('Ada
    Lovelace')
  },
})

cy.get('button').contains('Set name').click()
cy.get('[data-testid="greeting"]').should('have.text', 'Hello,
Ada Lovelace!')
cy.get('@promptDialog').should('have.been.calledWith', 'Enter
your name:')
```

To simulate the user cancelling the prompt (equivalent to `dialog.dismiss()` in Playwright), return `null` from the stub:

```
cy.visit('/', {
  onBeforeLoad(win) {
    cy.stub(win, 'prompt').returns(null)
  },
})
```

beforeunload

If your tests need to verify that a `beforeunload` handler is registered and sets a `returnValue`, use the `window:before:unload` event to assert on the event object directly:



Before: Playwright**test.ts**

```
page.on('dialog', async (dialog) => {
  expect(dialog.type()).toBe('beforeunload')
  await dialog.dismiss()
})

await page.close({ runBeforeUnload: true })
```

After: Cypress**test.cy.ts**

```
// Assert that the beforeunload handler fired and set a
returnValue
cy.on('window:before:unload', (event) => {
  expect(event.returnValue).to.eq('')
})

// Trigger navigation to fire beforeunload
cy.get('a').contains('Leave').click()
```

print dialogs

In Cypress, stub `window.print` using `onBeforeLoad` in `cy.visit()` and assert that the stub was called.

Before: Playwright**test.ts**

```
await page.goto('/')
await page.evaluate(
  '(() => { window.waitForPrint = new Promise(f => window.print = f) })()'
)
```



```
await page.getByRole('button', { name: 'Print invoice'
}).click()
await page.waitForFunction('window.waitForPrint')
```

After: Cypress

test.cy.ts

```
cy.visit('/', {
  onBeforeLoad(win) {
    cy.stub(win, 'print').as('printDialog')
  },
})

cy.get('button').contains('Print invoice').click()
cy.get('@printDialog').should('have.been.calledOnce')
```

API testing

Before: Playwright

test.ts

```
// APIRequestContext (Playwright)
const res = await request.get('/api/health')
expect(res.ok()).toBeTruthy()
```

After: Cypress

test.cy.ts

```
cy.request('/api/health').its('status').should('eq', 200)
```

See `cy.request()`.

Test generation



[Cypress Studio](#) lets you generate test code by interacting with your application in the Cypress app. It records actions like clicks and form inputs and writes the corresponding Cypress commands. Cypress Studio is a replacement for Playwright's `playwright codegen`. With a [Cypress Cloud](#) account, you also get **Studio AI**, AI-powered assertion recommendations as you record.

0:00 / 0:15

Component Testing

Playwright Component Testing is **experimental** and uses a `mount` fixture (for example via `@playwright/experimental-ct-react`). Cypress Component Testing uses `cy.mount()`, typically scaffolded in your Cypress support file.

Framework and bundler support

	Playwright Component Testing	Cypress Component Testing
Frameworks	Experimental packages for React, Vue, and Svelte	Official mounting libraries for React , Vue , Angular , and Svelte
Bundlers	Vite	Vite & Webpack

See [Component Testing](#) for more details.

Before: Playwright

button.test.ts



```
import { test, expect } from '@playwright/experimental-ct-react'
import { Button } from './Button'

test('renders', async ({ mount }) => {
  const component = await mount(<Button>Save</Button>)
  await expect(component).toContainText('Save')
})
```

After: Cypress

button.cy.ts

```
import { Button } from './Button'

describe('Button', () => {
  it('renders', () => {
    cy.mount(<Button>Save</Button>)
    cy.contains('button', 'Save').should('be.visible')
  })
})
```

See [Component Testing](#) and `cy.mount()`.

Reusable code patterns

Page object model

If you're using Page Object Model (POM) in Playwright, the same pattern works in Cypress:

Before: Playwright

loginPage.ts

```
export class LoginPage {
  constructor(private page: Page) {}
```




```
async login(username: string, password: string) {  
  await this.page.getByLabel('Username').fill(username)  
  await this.page.getByLabel('Password').fill(password)  
  await this.page.getByRole('button', { name: 'Login'  
}).click()  
}  
}
```

After: Cypress

support.cy.ts

```
export class LoginPage {  
  login(username: string, password: string) {  
    cy.get('[data-testid="username"]').type(username)  
    cy.get('[data-testid="password"]').type(password)  
    cy.contains('button', 'Login').click()  
  }  
}
```

Parameterized tests

Both Playwright and Cypress support generating tests from a data array using standard JavaScript. The pattern is the same in both tools: call `.forEach()` on your dataset outside of or inside a `describe` block, and generate one test per entry.

Before: Playwright

test.ts

```
:[  
  { name: 'Maya', expected: 'Hello, Maya!' },  
  { name: 'Theo', expected: 'Hello, Theo!' },  
  { name: 'Nora', expected: 'Hello, Nora!' },  
].forEach(({ name, expected }) => {  
  test(`testing with ${name}`, async ({ page }) => {  
    await page.goto(`https://example.com/greet?name=${name}`)
```



```
    await
    expect(page.getByRole('heading')).toHaveText(expected)
  })
})
```

After: Cypress

test.cy.ts

```
;[
  { name: 'Maya', expected: 'Hello, Maya!' },
  { name: 'Theo', expected: 'Hello, Theo!' },
  { name: 'Nora', expected: 'Hello, Nora!' },
].forEach(({ name, expected }) => {
  it(`testing with ${name}`, () => {
    cy.visit(`/greet?name=${name}`)
    cy.get('h1').should('have.text', expected)
  })
})
```

Custom commands

Before: Playwright

fixtures/auth.ts

```
export const test = base.extend({
  authenticatedPage: async ({ page }, use) => {
    await page.goto('/login')
    await page.getByLabel('Username').fill('user')
    await page.getByLabel('Password').fill('pass')
    await page.getByRole('button', { name: 'Login' }).click()
    await use(page)
  },
})
```

After: Cypress



support.cy.ts

```
Cypress.Commands.add('login', (username: string, password: string) => {
  cy.session([username, password], () => {
    cy.visit('/login')
    cy.get('[data-testid="username"]').type(username)
    cy.get('[data-testid="password"]').type(password)
    cy.contains('button', 'Login').click()
    cy.url().should('include', '/dashboard')
  })
})
```

See [Custom Commands](#) for more details.

CI parallelization

Key architectural difference

Playwright parallelizes tests locally using workers and across machines using manual sharding. You define the split strategy in config or at the command line.

- Worker count controlled via config (`workers: 4`) or CLI (`--workers=4`)
- Sharding splits tests across machines (`--shard=1/4`)
- You define the split strategy upfront

Before: Playwright

```
playwright test --shard=2/4 # Machine 2 playwright test --
shard=3/4 # Machine 3
playwright test --shard=4/4 # Machine 4
```

Cypress parallelizes tests across machines through [Cypress Cloud](#), which distributes specs dynamically based on historical run durations. This requires a Cypress Cloud account.

- Parallelization happens in CI through Cypress Cloud.



- Specs are distributed dynamically based on historical run data.
- No manual sharding required.

After: Cypress Cloud

```
cypress run--record --parallel
```

Smart Orchestration in Cypress Cloud

When you use Cypress Cloud, you gain **Smart Orchestration** features that go beyond simple parallelization:

Load Balancing

- Distributes specs across machines based on **previous run durations**
- Minimizes overall run time automatically
- No need to manually configure shards or splits

Spec Prioritization

- Runs specs that failed in the previous run first
- Provides faster feedback on fixes
- Reduces time to detect persistent failures

Auto Cancellation

- Cancels remaining specs after a threshold of failures
- Avoids wasting CI time on runs that can't pass
- Configurable failure thresholds

See [Parallelization](#) for setup details.

Debugging with Test Replay

Test Replay is available on all Cypress Cloud plans at no additional cost. It records a full, interactive replay of every test run, including network requests, console output, and DOM snapshots. replays are shareable via link and do not require a local trace file.



Test Replay Product Demo

Cypress.io



Xem trên

Before: Playwright

```
playwright test --trace on
```

```
npx playwright show-trace test-results/**/trace.zip
```

After: Cypress

```
cypress run --record --key <record_key>
```

See [Test Replay](#) for more details.

Handling flaky tests

Flaky tests are a common challenge in testing. Both frameworks offer retry mechanisms, but Cypress Cloud adds intelligent flake detection and management.

Before: Playwright

`playwright.config.ts`

```
export default {  
  retries: process.env.CI ? 2 : 0,
```



```
}
```

After: Cypress

cypress.config.ts

```
export default {  
  retries: {  
    runMode: 2,  
    openMode: 0,  
  },  
}
```

Flaky test management in Cypress Cloud

Beyond basic retries, Cypress Cloud provides:

Flaky test management

- Automatically identifies tests that pass after retry
- Tracks flake rate over time
- Provides analytics on which specs are most flaky
- Helps prioritize stability improvements

Accessibility testing

If you record test runs to Cypress Cloud, [Cypress Accessibility](#) provides automated accessibility scanning across your entire test suite with no changes to your test code.

Rather than requiring `checkA11y()` calls in each test, Cypress Accessibility captures DOM snapshots during test execution server-side, using the same protocol as Test Replay. Every distinct state your tests interact with is scanned automatically. This means pages and components that your tests visit but do not explicitly instrument are still checked.

Learn more about [Accessibility testing](#).



Cheat sheet

The `await` required in Playwright is not included in the cheat sheet for readability.

Playwright	
<code>page.locator('css')</code>	<code>cy.get('css')</code>
<code>page.getByTestId('submit')</code>	<code>cy.get('[data-t</code>
<code>page.goto('/path')</code>	<code>cy.visit('/path</code>
<code>page.reload()</code>	<code>cy.reload()</code>
<code>page.title()</code>	<code>cy.title()</code>
<code>page.url()</code>	<code>cy.url()</code>
<code>page.waitForResponse(...)</code>	<code>cy.intercept(..</code>
<code>page.setInputFiles(selector, 'file.png')</code>	<code>cy.get(selector</code>
<code>page.locator(selector).click()</code>	<code>cy.get(selector</code>
<code>page.locator(selector).click({ force: true })</code>	<code>cy.get(selector</code>
<code>page.locator(selector).click({ button: 'right' })</code>	<code>cy.get(selector</code>
<code>page.locator(selector).click({ modifiers: ['Shift'] })</code>	<code>cy.get(selector</code>
<code>page.locator(selector).dblclick()</code>	<code>cy.get(selector</code>
<code>page.locator(selector).hover()</code>	<code>cy.get(selector</code>



Playwright	
<code>page.locator(selector).fill('text')</code>	<code>cy.get(selector)</code>
<code>page.locator(selector).focus()</code>	<code>cy.get(selector)</code>
<code>page.locator(selector).press('Tab')</code>	<code>cy.get(selector)</code>
<code>page.locator(selector).selectOption('blue')</code>	<code>cy.get('[data-t</code>
<code>page.locator(selector).scrollIntoViewIfNeeded()</code>	<code>cy.get(selector)</code>
<code>page.locator(selector).screenshot()</code>	<code>cy.get(selector).scre</code>
<code>page.route('**/api', ...)</code>	<code>cy.intercept('/</code>
<code>page.screenshot()</code>	<code>cy.screenshot()</code>
<code>use: { storageState: 'state.json' }</code>	<code>cy.session('use</code>
<code>page.clock.setFixedTime(date)</code>	<code>cy.clock(now)</code>
<code>page.clock.install({ time })</code>	<code>cy.clock(now)</code>
<code>page.clock.fastForward(duration)</code>	<code>cy.tick(ms)</code>
<code>page.clock.runFor(duration)</code>	<code>cy.tick(ms)</code>
<code>page.clock.setSystemTime(date)</code>	<code>clock.setSystem</code>
<code>page.on('dialog', dialog => dialog.accept())</code>	Auto-accepted by
<code>page.on('dialog', dialog => dialog.dismiss())</code>	<code>cy.on('window:c</code>
<code>page.on('dialog', dialog => dialog.accept('x'))</code>	<code>cy.stub(win, 'p onBeforeLoad</code>



Playwright	
<code>page.on('dialog') with beforeunload</code>	<code>cy.on('window:b</code>
<code>expect(page).toHaveURL(/re/)</code>	<code>cy.url().should</code>
<code>expect(locator).toHaveText('Yes!')</code>	<code>cy.get(selector</code>
<code>expect(locator).toHaveAttribute('href')</code>	<code>cy.get(selector</code>
<code>expect(locator).toBeChecked()</code>	<code>cy.get(selector</code>
<code>expect(locator).toBeDisabled()</code>	<code>cy.get(selector</code>
<code>expect(locator).toBeVisible()</code>	<code>cy.get(selector</code>
<code>expect(locator).toBeHidden()</code>	<code>cy.get(selector</code>
<code>expect(locator).toBeEnabled()</code>	<code>cy.get(selector</code>
<code>expect(locator).toBeAttached()</code>	<code>Cypress.dom.isA</code>
<code>expect(locator).toBeFocused()</code>	<code>cy.get(selector</code>
<code>expect(locator).toHaveValue('123')</code>	<code>cy.get(selector</code>
<code>expect(locator).toContainText('Yes!')</code>	<code>cy.get(selector</code>
<code>expect(locator).toContainClass('light')</code>	<code>cy.get(selector</code>
<code>expect(locator).toHaveCSS('color', 'red')</code>	<code>cy.get(selector 'red')</code>
<code>browserContext.cookies()</code>	<code>cy.getCookies()</code>
<code>browserContext.clearCookies()</code>	<code>cy.clearCookies</code>



Playwright	
<code>browserContext.addCookies(cookie)</code>	<code>cy.setCookie(name, value, url)</code>

Last updated on **Mar 4, 2026**

CONTENTS

